Universidad de Buenos Aires

Facultad de Ciencias Exactas y Naturales

TP2: AirMiles

Grupo JavaConMates

Integrantes:

Ayala Ignacio
Della Bonzana Lara
Morales Santiago
Vannelli Tiziana

24 de octubre de 2026

1. Introducción

En este documento se justifican las estructuras de datos elegidas para implementar el TAD `AirMiles`. Para cada estructura se describe la información que almacena, las operaciones que permite realizar y su relación con las complejidades exigidas por la especificación.

Denotamos por U la cantidad de usuarios registrados y por $|L|$ la longitud del nombre de una promoción.

El objetivo es seleccionar estructuras que permitan implementar las operaciones requeridas respetando sus complejidades temporales, tanto en el caso promedio como en el peor caso, según corresponda.

2. Almacenamiento de usuarios

2.1. Estructura elegida

Para almacenar los usuarios se utiliza un árbol binario de búsqueda (ABB). Cada elemento del árbol es un objeto de tipo `Usuario`, ordenado según su identificador único (`id`) mediante la implementación de `Comparable<Usuario>`.

El ABB mantiene referencias a los hijos izquierdo y derecho de cada nodo, así como a su padre.

2.2. Justificación

La elección de un ABB permite buscar e insertar usuarios utilizando su identificador. Además, permite recorrer todos los usuarios mediante un iterador.

Al tratarse de un ABB sin balanceo automático, las operaciones de búsqueda e inserción tienen complejidad $O(\log U)$ en el caso promedio y $O(U)$ en el peor caso. Esto coincide con las cotas establecidas para las operaciones de usuarios en la especificación.

2.3. Operaciones asociadas

El ABB se utiliza en las siguientes operaciones:

- `registrarUsuario` y `registrarUsuarioConReferidor`: insertar usuarios.
- `consultarSaldo` y `consultarCategoria`: buscar usuarios por ID.
- `acumularMillas`, `canjearMillas` y `transferirMillas`: localizar las cuentas que deben actualizarse.
- `paresTransferidoresExclusivos`: recorrer todos los usuarios.

3. Representación de los usuarios

Cada usuario se representa mediante un objeto `Usuario`, que agrupa la información correspondiente a su cuenta.

Entre los atributos considerados se encuentran:

- Identificador del usuario.
- Millas disponibles.
- Millas acumuladas históricamente.
- Millas canjeadas históricamente.
- Historial de transacciones.
- Información auxiliar para las relaciones de transferencia con otros usuarios.

Agrupar estos datos en un objeto permite acceder a la información de una cuenta una vez localizado el usuario en el ABB. Las modificaciones se realizan sobre el mismo objeto almacenado en el árbol, sin necesidad de reemplazarlo.

4. Historial de transacciones

4.1. Estructura elegida

El historial de cada usuario se representa mediante una lista enlazada. Se mantiene una referencia al último nodo de la lista para poder agregar nuevas transacciones al final.

4.2. Justificación

La lista enlazada permite conservar las transacciones en el orden en que se realizan. Mantener una referencia al último nodo permite agregar una transacción en tiempo $O(1)$.

Cada transacción almacena los datos necesarios para representar la operación, como su tipo y cantidad de millas. Cuando corresponda, también incluye la distancia, la promoción, el factor aplicado y los usuarios involucrados.

5. Promociones

5.1. Estructura elegida

Para almacenar las promociones se considera utilizar un trie (árbol de prefijos), en el que cada camino desde la raíz representa una secuencia de caracteres del nombre de una promoción.

El nodo correspondiente al final de cada nombre almacena la información de la promoción, como su factor y, si corresponde, su tope y cupo restante.

5.2. Justificación

Si el acceso a los hijos de cada nodo tiene costo constante, buscar, insertar y eliminar una promoción requiere recorrer sus caracteres. Por lo tanto, estas operaciones tienen complejidad $O(|L|)$, donde $|L|$ es la longitud del nombre.

Esta estructura resulta adecuada porque las complejidades especificadas para iniciar y finalizar promociones dependen de la longitud del nombre y no de la cantidad total de promociones.

La implementación debe contemplar todos los caracteres permitidos en los nombres de promoción.

6. Máximos históricos

Las operaciones `elMasAcumulador` y `elMasCanjeador` deben responder en tiempo $O(1)$.

Para cumplir esta exigencia, se mantienen referencias o identificadores de los usuarios que poseen, respectivamente, la mayor cantidad histórica de millas acumuladas y la mayor cantidad histórica de millas canjeadas.

Cada vez que un usuario acumula o canjea millas, se actualiza su contador histórico y se verifica si corresponde actualizar alguno de los máximos.

De esta manera, las consultas no necesitan recorrer todos los usuarios. La implementación debe definir cómo resolver los empates y mantener los máximos actualizados durante todas las operaciones.

7. Ranking de millas disponibles

7.1. Estructura elegida

Para obtener los N usuarios con mayor cantidad de millas disponibles se utiliza un max-heap ordenado por saldo disponible.

Cada elemento representa un usuario y la raíz contiene al usuario con mayor saldo. Además, se mantiene información auxiliar que permite localizar la posición de cada usuario en el heap cuando es necesario actualizar su prioridad.

7.2. Justificación

La propiedad del max-heap garantiza que cada padre tiene una prioridad mayor o igual que la de sus hijos. No es necesario que el hijo izquierdo sea mayor que el derecho.

Cada extracción del máximo cuesta $O(\log U)$. Por lo tanto, obtener los primeros N usuarios cuesta

$$O(N \log U),$$

como requiere la especificación.

Cuando una operación modifica el saldo disponible de un usuario, debe actualizarse su posición en el heap para preservar la propiedad de max-heap.

8. Pares de transferidores exclusivos

8.1. Objetivo

La operación `paresTransferidoresExclusivos` devuelve todos los pares de usuarios que han transferido millas exclusivamente entre sí. Su complejidad requerida es $O(U)$.

8.2. Representación de las relaciones

Para resolver esta consulta se mantiene información auxiliar sobre las contrapartes de cada usuario. Una contraparte es otro usuario con quien se registró al menos una transferencia, ya sea enviada o recibida.

La representación debe distinguir usuarios diferentes y evitar contar varias veces a una misma contraparte cuando se repiten las transferencias.

8.3. Estrategia de consulta

La consulta recorre todos los usuarios una sola vez y utiliza la información auxiliar mantenida durante las transferencias para identificar aquellos que cumplen la condición de exclusividad.

Cada par se agrega una única vez. Por ejemplo, se puede imponer que el identificador del primer usuario sea menor que el del segundo.

Para garantizar $O(U)$ en el peor caso, la consulta debe realizar trabajo constante por usuario y evitar búsquedas adicionales en el ABB para localizar a cada contraparte. Por lo tanto, la estructura auxiliar debe permitir verificar la relación de exclusividad en tiempo constante.

9. Resumen de complejidades

La siguiente tabla resume las complejidades requeridas por la especificación.

Operación	**Complejidad requerida**
Crear el sistema	$O(1)$
Registrar usuarios	$O(\log U)$ promedio, $O(U)$ peor caso
Acumular millas	$O(\log U + \|L\|)$ promedio, $O(U + \|L\|)$ peor caso
Canjear millas	$O(\log U)$ promedio, $O(U)$ peor caso
Transferir millas	$O(\log U)$ promedio, $O(U)$ peor caso
Consultar saldo o categoría	$O(\log U)$ promedio, $O(U)$ peor caso
Iniciar o finalizar promociones	$O(\|L\|)$
Máximo acumulador o canjeador	$O(1)$
Ranking de millas	$O(N \log U)$
Pares de transferidores exclusivos	$O(U)$

10. Conclusión

Las estructuras seleccionadas buscan equilibrar la facilidad de implementación con las complejidades exigidas por la especificación.

El ABB permite organizar los usuarios por identificador; el trie permite operar sobre promociones en función de la longitud de sus nombres; la lista enlazada facilita registrar transacciones; y el max-heap permite obtener eficientemente los usuarios con mayor saldo disponible.

Las consultas de máximos históricos y de transferidores exclusivos requieren mantener información auxiliar actualizada durante las operaciones que modifican el sistema. La corrección de estas actualizaciones es fundamental para garantizar tanto los resultados como las complejidades requeridas.